An Empirical Analysis of Entangled Enhancement in an Autonomous Driving Pipeline

***Abstract*—Entangled enhancement is a defective retraining of a machine learning (ML) model that improves the updated model but fails to improve the whole system. Although the problem is known in the literature, little has been studied in real ML pipelines. This study empirically investigates real entangled enhancements in an autonomous driving pipeline consisting of perception, prediction and planning modules. We train an ML pipeline of three models on the nuScenes data set. We then investigate the change in the prediction module when the perception model is updated with a new data set. We test 30 different updates, which differ in the amount of new data, the number of training epochs, the random seed, and the initial perception model. Entangled enhancement is observed in the prediction module in 23 of the 30 updates. To analyze the cause, we conduct a comparative study, in which the prediction model is trained either on the ground truth or on the output of the perception model. An increase larger than the variation caused by a different random seed appears in 11 updates after training on the output of the perception model. No such increase appears after training on the ground truth. This result indicates that the adaptation of the prediction model to the errors of the perception model causes entangled enhancement.**

***Index Terms*—autonomous driving, entangled enhancement, machine learning system, model maintenance, retraining**

I. Introduction

A Machine Learning System (MLS) is often built as a pipeline of ML models sequentially connected, where each model uses the output of the preceding model. A model earlier in the sequence is upstream, and a model later in the sequence is downstream. Retraining an upstream model can improve that model and still fail to improve the whole system. Wang and Machida call this defecting update as *entangled enhancement* [1], [2].

Several studies describe the risk of entangled enhancement. Sculley et al. [3] and Wu et al. [4] show that a change to one model of an MLS can make the whole system worse. Xu et al. [5] show that the prediction error increases when the input of the prediction model comes from a real perception model instead of the ground truth, which consists of the objects labelled by hand. However, none of these studies examined the significance of entangled enhancement in a real autonomous driving pipeline, consisting of perception, prediction, and planning modules.

In this study, we empirically investigate whether an update that improves the perception model increases the error of the prediction and planning models in an autonomous driving pipeline. When increased errors are observed, we further investigate the impact of adaptation of the downstream models. Adaptation means that the prediction and planning models, trained on the output of the perception model, learn to work with the errors of the perception model.

We train an ML pipeline on the nuScenes data set. We then update the perception model 30 times with a new data set. The 30 updates are not repetitions of one procedure. They differ in the amount of new data, the number of training epochs, the random seed, and the initial perception model. To analyze the impact of adaptation, we compare prediction and planning models trained on the ground truth with prediction and planning models trained on the output of the perception model.

We observe that the prediction error increases in 23 of the 30 updates after training on the output of the perception model. As a result, we confirm that entangled enhancement occurs in the prediction model. After training on the ground truth, the prediction error increases in only 5 of the 30 updates. None of these 5 increases is larger than the variation caused by a different random seed. A prediction model trained on the ground truth cannot adapt to the errors of the perception model. The increase after training on the output of the perception model therefore comes from adaptation.

II. Related Work

Studies on the maintenance of ML systems describe how a change to one model affects the whole system. Sculley et al. [3] describe how a change to one model of an MLS changes the behaviour of every model that uses the changed output. Wu et al. [4] show that improving one model of a system can make the whole system worse. Wang and Machida [1], [2] model when an MLS should retrain each of its models, using the probability that a retraining causes entangled enhancement. In this study, we examine the adaptation of the downstream models as a cause of entangled enhancement. The downstream models adapt during training to the errors of the upstream model. The retraining then changes those errors, which disturbs the downstream models.

Studies on model updates examine the errors that an update introduces into a single model. Yan et al. [6] show that an updated classifier can make errors on inputs that the previous classifier handled correctly, even when the updated classifier is more accurate overall. They train the updated model to reduce these errors. Shen et al. [7] train an updated model whose feature representations remain compatible with those of the previous model. These studies consider one model and do not measure the effect of an update on downstream models.

Studies on autonomous driving pipelines examine how the errors of the perception model affect prediction. Xu et al. [5]

show that the prediction error increases when the input comes from a real perception model instead of the ground truth. Ivanovic et al. [8] carry the uncertainty of the perceived object positions into the prediction of object paths. Weng et al. [9] make a prediction model robust to the errors of object tracking, which links the same object across images. These studies keep the perception model fixed. In contrast, this study updates the perception model of a pipeline in which all three modules are ML models. We then measure the change of the prediction and planning errors after each update.

III. Methodology

This section defines the target pipeline and the notation, and states the research questions of this study.

A. Target Pipeline and Notation

This study focuses on an autonomous driving pipeline that consists of a perception module, a prediction module and a planning module. We call this pipeline the target pipeline. We write C_1 for the perception model, C_2 for the prediction model and C_3 for the planning model. C_1 finds the objects in the image of the front camera. C_2 predicts the path of each object over the next 3 s. C_3 chooses the path of the ego vehicle over the same 3 s. An update is one retraining of C_1 on new data. We write C_1' for the updated C_1.

We write δ_1 for the gain of C_1 in mAP50, the standard accuracy measure for object detection. The prediction error is the minADE, which is the average distance between the predicted path and the true path of an object, taken for the best of the paths that C_2 predicts. The nuScenes data set records the path of the ego vehicle driven by a human driver. We use this recorded path as the ground truth for planning. The planning error is the distance between the planned position and the recorded position at 3 s. We write Δ_2 and Δ_3 for the increase of the prediction error and of the planning error after an update, respectively. An update causes entangled enhancement when

$$\delta_1 > 0 \quad \text{and} \quad (\Delta_2 > 0 \text{ or } \Delta_3 > 0). \tag{1}$$

B. Research Questions

The goal of this study is to confirm whether entangled enhancement occurs in the target pipeline and to find its cause. We ask two research questions.

RQ1: Does an update that improves C_1 increase the error of C_2 or C_3? The studies in Section II describe the risk that an update of an upstream model increases the error of the downstream models. However, none of these studies examined this risk in a real autonomous driving pipeline. To answer RQ1, we update C_1 and measure Δ_2 and Δ_3 in the pipeline.

RQ2: Does the increase come from the adaptation of C_2 and C_3 to the errors of C_1? Only training on the output of C_1 lets C_2 and C_3 learn the errors of C_1, which an update then changes. To answer RQ2, we compare C_2 and C_3 trained on the output of C_1 with C_2 and C_3 trained on the ground truth.

IV. Experimental Setup

This section presents the data set, the implementation of the pipeline, the updates of C_1, and the experiments for RQ1 and RQ2. Fig. 1 shows the order of the experiments. We ran all experiments in a Kaggle notebook with an Intel Xeon CPU at 2.0 GHz (4 virtual cores), 31 GB of RAM, Ubuntu 24.04 and two NVIDIA Tesla T4 GPUs with 15 GB of memory each (Kaggle GPU T4 x2). Training used one of the two GPUs. The software is Python 3.13, PyTorch 2.11 with CUDA 12.8, and Ultralytics 8.4.

A. Data Set and Its Preparation

We use 350 of the 850 scenes of nuScenes [10], which are driving clips of about 20 s from Boston and Singapore. We use the full-size images of the front camera only (4.6 GB). A keyframe is an image with labelled objects, recorded twice per second. We split the 350 scenes into four parts. Boston 1 (100 scenes) trains C_1. Boston 2 (100 scenes) trains C_2 and C_3, with 14 of the scenes kept for validation. C_1 never sees Boston 2 during training. The output of C_1 on Boston 2 therefore contains the errors that C_1 makes on unseen data [5]. Singapore 1 (100 scenes) provides the new data for the updates. Singapore 2 (50 scenes, 2,020 images) is the test set. The test set has 1,720 test samples, one for every keyframe with 3 s of future.

B. Implementation of the Pipeline

We use YOLOv11n [11] as a perception model (C_1), which is trained for 10 epochs on Boston 1, where an epoch is one pass over the training data. We train two initial C_1 with the random seeds 0 and 1, where a random seed is the starting number for the random choices made during training. The two initial C_1 reach an mAP50 of 0.109 and 0.117 on Singapore 2.

AutoBot-Ego [12] is employed as a prediction model (C_2). For each object in the current keyframe, a transformer reads the objects of the last four keyframes relative to that object and outputs six possible paths of 3 s. C_2 needs no hand-written rule that links objects across images [9].

For the planning model (C_3), we employ PlanT [13]. C_3 receives one input for each object that C_1 finds. This input holds the current position of the object, the most likely path of the object predicted by C_2, the class of the object, and the confidence of C_1 in the object. Two further inputs hold the driving command and the path of the ego vehicle over the last 1.5 s, which we call the ego path. The command is left, straight or right [14]. A recurrent decoder outputs six points of the planned path over 3 s.

We train C_2 and C_3 on the output of C_1 on Boston 2, once for each initial C_1. We call this training mode pipe training, because the training input of C_2 and C_3 comes from C_1, as in the pipeline. Training stops early when the error on the validation scenes stops decreasing. The training data are augmented by mirroring and rotation. A planning module that receives the ego path can ignore the objects [15]. We check whether C_3 uses the objects by removing the objects from the input of C_3. The removal moves the planned position at 3 s by

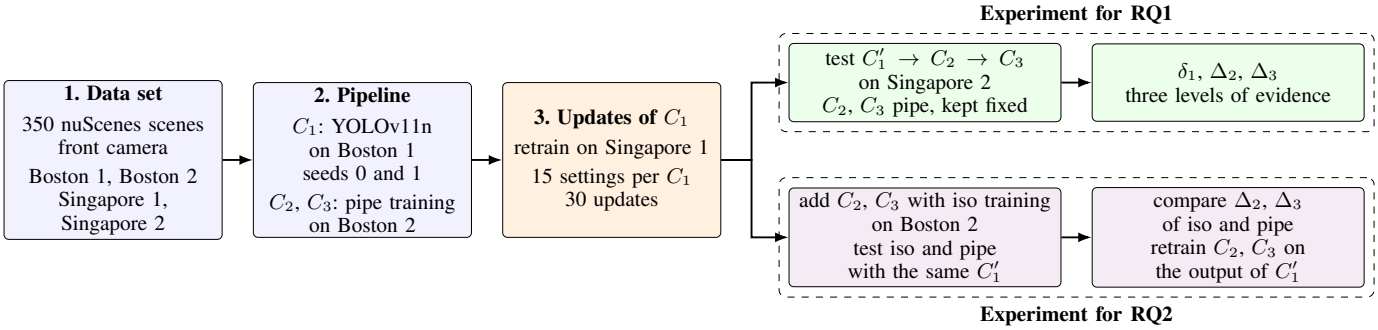


Fig. 1. Order of the experiments. The experiment for RQ1 updates C_1 only. The experiment for RQ2 adds iso training of C_2 and C_3.

Algorithm 1: Update procedure for one initial C_1 and one training mode

Input : initial C_1, training mode $r \in \{\text{pipe}, \text{iso}\}$, settings $\mathcal{S}$, Boston 2, Singapore 1, Singapore 2
Output: δ_1, Δ_2 and Δ_3 for each setting

1 Train C_2^r, C_3^r on Boston 2 in the training mode r
2 $E_2, E_3 \leftarrow$ errors of $C_1 \rightarrow C_2^r \rightarrow C_3^r$ on Singapore 2
3 **foreach** *setting* $s \in \mathcal{S}$ **do**
4 $\quad C_1' \leftarrow$ retrain C_1 on Singapore 1 with s
5 $\quad \delta_1 \leftarrow \text{mAP50}(C_1') - \text{mAP50}(C_1)$ on Singapore 2
6 $\quad \Delta_2, \Delta_3 \leftarrow$ errors of $C_1' \rightarrow C_2^r \rightarrow C_3^r$ on Singapore 2 minus E_2, E_3

0.73 to 1.65 m on average. This change is above the threshold of 0.5 m, which we fixed before the experiment.

C. Updates of the Perception Model

An update retrains one initial C_1 on Singapore 1 with one of 15 settings. The settings differ in the amount of new data, the number of epochs and the random seed. Nine settings use 100 %, 50 % or 25 % of the images with 10 epochs. Six settings use all images with 5 or 20 epochs. Each of these five combinations of data and epochs is used with the seeds 1, 2 and 3. The 15 settings applied to both initial C_1 give 30 updates. The 30 updates are not repetitions of one procedure. The updates differ in the amount of new data, the number of epochs, the random seed and the initial C_1. The test scenes are never used to choose a model.

Algorithm 1 gives the procedure for one initial C_1 and one training mode r of C_2 and C_3. Running the procedure for both initial C_1 gives the 30 updates. In Algorithm 1, 'pipe' indicates pipe training, and 'iso' indicates iso training, in which C_2 and C_3 are trained on the ground truth (Section IV-E). Line 1 trains C_2 and C_3 on Boston 2 in the training mode r. Line 2 measures the prediction error E_2 and the planning error E_3 of the pipeline with C_1 on Singapore 2. For each setting s, line 4 retrains C_1 on Singapore 1 and gives C_1'. Line 5 computes the gain δ_1. Line 6 replaces C_1 with C_1', keeps C_2 and C_3 fixed, and computes Δ_2 and Δ_3 against E_2 and E_3.

D. Experiment for RQ1

To answer RQ1, we run Algorithm 1 with pipe training for both initial C_1. C_2 and C_3 stay fixed during all 30 updates. The prediction error is computed over the objects found by C_1 that are matched one to one with a ground truth object of the same class. The outputs of C_2 and C_3 on the ground truth stay exactly the same before and after every update, which confirms that only the output of C_1 changes.

An increase can also come from randomness alone. The two initial C_1 have almost the same accuracy. Replacing one initial C_1 with the other still changes the errors of C_2 and C_3, although neither C_1 is better. We call the largest of these changes the seed variation. The seed variation is $\tau_2 = 0.112\,\text{m}$ for the prediction error and $\tau_3 = 0.018\,\text{m}$ for the planning error. We count the updates with an increase at three levels of evidence. The first level counts any increase. The second level counts an increase above the seed variation. The third level counts an increase whose 95 % bootstrap interval lies above zero. The bootstrap repeats the measurement on 2,000 test sets, each drawn from the 50 test scenes with replacement.

E. Experiment for RQ2

To answer RQ2, we add a second training mode. In iso training, we train C_2 and C_3 on the ground truth of Boston 2, isolated from C_1. Only pipe training lets C_2 and C_3 learn the errors of C_1. We run Algorithm 1 with iso training for both initial C_1 and the same 30 updates. In each update, the iso and pipe versions of C_2 and C_3 receive the same output of C_1'. If the increase comes from adaptation, Δ_2 and Δ_3 are larger after pipe training than after iso training. We count the increases at the same three levels of evidence.

We also retrain C_2 and C_3 on the output of C_1' for two further updates, one from each initial C_1 with all images and 10 epochs. The retraining lets C_2 and C_3 adapt to the errors of C_1'. If adaptation causes the increase, the retraining lowers the prediction error and the planning error.

V. Results and Discussion

This section answers RQ1 and RQ2 in order. In all 30 updates, δ_1 lies between $+0.048$ and $+0.118$. Every update therefore improves C_1. An update causes entangled enhancement whenever $\Delta_2 > 0$ or $\Delta_3 > 0$.

A. Results for RQ1

Fig. 2 shows Δ_2, the increase of the prediction error, for every update. The red circles show the pipeline of the

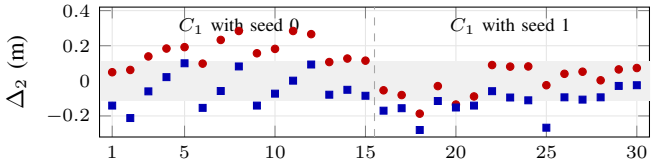


Fig. 2. Δ_2, the increase of the prediction error, in the 30 updates. Red circles show pipe training, blue squares show iso training, and the grey band shows the seed variation. Updates 1 to 15 start from the initial C_1 with seed 0.

TABLE I
UPDATES, OUT OF 30, WITH AN INCREASE OF THE ERROR

	Prediction Δ_2		**Planning Δ_3**	
Level of evidence	**pipe**	**iso**	**pipe**	**iso**
Any increase	23	5	18	29
Above seed variation	11	0	13	29
Bootstrap interval above zero	3	0	1	10

experiment for RQ1, in which C_2 and C_3 use pipe training. Table I counts the updates with an increase at the three levels of evidence.

The prediction error increases in 23 of the 30 updates (Table I, columns "pipe"). In 11 updates, the increase is larger than the seed variation. Randomness alone does not explain these 11 increases. All 11 updates start from the initial C_1 with seed 0 (left half of Fig. 2). For this C_1, every update increases the prediction error, by $+0.049$ to $+0.284$ m with a mean of $+0.165$ m. For the initial C_1 with seed 1, 8 of the 15 updates increase the prediction error, but none by more than the seed variation. With 50 test scenes, the bootstrap interval of Δ_2 spans about ± 0.25 m. Only 3 increases have an interval above zero.

These values show that entangled enhancement occurs in the prediction model of the pipeline. An update that improves C_1 can increase the prediction error beyond the change caused by a different random seed. The size of the increase depends more on the initial C_1 than on the setting of the update.

The planning error increases in 18 of the 30 updates, 13 of them above the seed variation of 0.018 m. The increases are small. Δ_3 is at most 0.072 m, against a planning error of about 4 m before the update. Only 1 increase has a bootstrap interval above zero. The planning error changes little because the objects affect C_3 only weakly. Removing all objects from the input of C_3 moves the planned position at 3 s by only 0.73 to 1.65 m (Section IV-B). An update changes only some of the objects that C_1 finds.

Answer to RQ1: An update that improves C_1 increases the prediction error in 23 of the 30 updates, 11 of them beyond the seed variation. The planning error increases in 18 updates, by at most 0.072 m.

B. Results for RQ2

In all 30 updates, Δ_2 after pipe training is larger than Δ_2 after iso training (red circles above blue squares in Fig. 2). After iso training, the prediction error increases in only 5 updates. None of these increases exceeds the seed variation (Table I). Both versions of C_2 receive the same output of C_1'. Only the version trained on the output of C_1 has learned the errors of C_1. Only this version increases its error beyond the seed variation. The comparison results characterize the cause of the increase of the prediction error. The increase comes from the adaptation of C_2 to the errors of C_1, which the update changes.

For the planning error, the comparison gives the opposite result. The increase appears more often after iso training (29 of 30 updates) than after pipe training (18 of 30). Adaptation cannot explain an increase after iso training. The C_3 trained on the ground truth receives the predicted paths of C_2 only at test time. A change in the objects that C_1' finds can therefore affect this C_3 more than the C_3 trained on the output of C_1. These increases are also small, at most 0.127 m.

The retraining of C_2 and C_3 on the output of C_1' (Section IV-E) tests the same cause from the other side. After pipe training, the two retrained updates give a prediction error of 2.75 and 2.61 m. The planning error is 4.14 and 4.00 m. After the retraining, the prediction error falls to 2.39 and 2.45 m. The planning error falls to 3.71 and 3.70 m. The retraining lowers the prediction error by 0.36 and 0.16 m and the planning error by 0.44 and 0.30 m. Both errors fall below the errors before the update (2.62 and 2.66 m, 4.16 and 4.00 m). When C_2 and C_3 adapt to the errors of C_1', the update becomes an improvement of the whole pipeline. This result supports the retraining of downstream models that Wang and Machida model [1], [2].

Answer to RQ2: For the prediction error, yes. The increase appears only when C_2 has learned the errors of C_1. The retraining on the output of C_1' removes the increase. For the planning error, adaptation does not explain the small increase, because the increase appears more often after iso training.

VI. CONCLUSION

In this study, we investigated the entangled enhancement in a real autonomous driving pipeline. The pipeline consists of a perception model (YOLOv11n), a prediction model (AutoBot-Ego) and a planning model (PlanT), trained on the nuScenes data set. We updated the perception model 30 times with new data. The 30 updates differ in the amount of new data, the number of training epochs, the random seed and the initial perception model.

To answer RQ1, we measured the prediction error and the planning error after each update. Entangled enhancement occurs in the prediction model in 23 of the 30 updates. In 11 of these updates, the increase of the prediction error is larger than the seed variation. To answer RQ2, we compared the prediction and planning models trained on the output of the perception model with the models trained on the ground truth. An increase of the prediction error above the seed variation appears only after training on the output of the perception model. Retraining the prediction and planning models on the output of the updated perception model lowers both errors below their values before the update. These results indicate

that the adaptation of the prediction model to the errors of the perception model causes entangled enhancement. As future work, we develop a method that detects entangled enhancement after an update of the perception model, before ground truth for the new data exists.

REFERENCES

[1] Z. Wang and F. Machida, "Maintaining performance of a machine learning system against imperfect retraining," in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 1782–1787.

[2] Z. Wang and F. Machida, "Exploiting the availability-continuity trade-off in imperfect retraining of machine learning systems," in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 406–417.

[3] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 2503–2511.

[4] R. Wu, C. Guo, A. Hannun, and L. van der Maaten, "Fixes that fail: Self-defeating improvements in machine-learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 11 745–11 756.

[5] Y. Xu, L. Chambon, É. Zablocki, M. Chen, A. Alahi, M. Cord, and P. Pérez, "Towards motion forecasting with real-world perception inputs: Are end-to-end approaches competitive?" in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 18 428–18 435.

[6] S. Yan, Y. Xiong, K. Kundu, S. Yang, S. Deng, M. Wang, W. Xia, and S. Soatto, "Positive-congruent training: Towards regression-free model updates," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 14 299–14 308.

[7] Y. Shen, Y. Xiong, W. Xia, and S. Soatto, "Towards backward-compatible representation learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6368–6377.

[8] B. Ivanovic, Y. Lin, S. Shrivastava, P. Chakravarty, and M. Pavone, "Propagating state uncertainty through trajectory forecasting," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.

[9] X. Weng, B. Ivanovic, K. Kitani, and M. Pavone, "Whose track is it anyway? Improving robustness to tracking errors with affinity-based trajectory prediction," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[10] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, "nuScenes: A multimodal dataset for autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 621–11 631.

[11] G. Jocher and J. Qiu, "Ultralytics YOLO11," https://github.com/ultralytics/ultralytics, 2024.

[12] R. Girgis, F. Golemo, F. Codevilla, M. Weiss, J. A. D'Souza, S. E. Kahou, F. Heide, and C. Pal, "Latent variable sequential set transformers for joint multi-agent motion prediction," in *International Conference on Learning Representations (ICLR)*, 2022.

[13] K. Renz, K. Chitta, O.-B. Mercea, A. S. Koepke, Z. Akata, and A. Geiger, "PlanT: Explainable planning transformers via object-level representations," in *Conference on Robot Learning (CoRL)*, ser. PMLR, vol. 205, 2022.

[14] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, "Rethinking the open-loop evaluation of end-to-end autonomous driving in nuScenes," *arXiv preprint arXiv:2305.10430*, 2023.

[15] Z. Li, Z. Yu, S. Lan, J. Li, J. Kautz, T. Lu, and J. M. Alvarez, "Is ego status all you need for open-loop end-to-end autonomous driving?" in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.